

LLD Design Template

Low-Level Design, or LLD, is the process of converting requirements into a clear object-oriented design. In placements, LLD questions test whether you can think in terms of classes, responsibilities, relationships, APIs, edge cases, and maintainable code.

This template is designed for interview practice. You can use it for common problems like Parking Lot, Elevator System, Splitwise, BookMyShow, Chess, ATM, Vending Machine, Library Management System, Logging Framework, Rate Limiter, Food Delivery, Ride Sharing, and Hotel Management.

How to Use This Template in an Interview

When an interviewer gives an LLD problem, do not jump directly to classes. A strong answer usually follows this flow:

1. Clarify requirements and constraints.
2. Define the scope.
3. Identify actors and use cases.
4. Identify entities and relationships.
5. Assign responsibilities.
6. Define classes, fields, methods, and APIs.
7. Discuss design patterns if useful.
8. Apply SOLID principles.
9. Handle edge cases.
10. Mention extension points and tradeoffs.

You do not need to write production-perfect code in the first attempt. The interviewer mainly wants to see whether your design is clean, extensible, and easy to reason about.

1. Problem Statement

Write the problem in your own words.

Example:

```
Design a Parking Lot system that allows vehicles to enter, park in available slots, generate tickets, calculate parking fees, and exit after payment.
```

Interview Tip

Restating the problem helps confirm that you and the interviewer are solving the same thing.

2. Clarify Requirements

Requirements are the expected features of the system. In LLD, you should separate them into functional and non-functional requirements.

Functional Requirements

Functional requirements describe what the system should do.

Ask questions like:

- What are the main actions supported by the system?
- Who will use the system?
- What operations should be available?
- What is in scope and what is out of scope?
- Should the system support create, read, update, and delete operations?
- Should the system support search, filter, sort, or booking?
- Should payment, notification, authentication, or analytics be included?

Template:

The system should allow:

1. ...
2. ...
3. ...

Example for Parking Lot:

The system should allow:

1. Vehicles to enter the parking lot.
2. The system to assign an available parking spot.
3. The system to generate a parking ticket.
4. Vehicles to exit using the ticket.
5. The system to calculate parking fees.
6. The system to mark the spot as available after exit.

Non-Functional Requirements

Non-functional requirements describe how the system should behave.

Common LLD non-functional concerns:

- Maintainability
- Extensibility
- Readability
- Testability
- Low coupling
- High cohesion
- Thread safety
- Performance
- Scalability
- Fault tolerance

- Data consistency

Example:

The design should be extensible so that new vehicle types, fee strategies, and parking spot types can be added without changing core logic.

Out of Scope

Mention what you are not designing unless the interviewer asks.

Example:

Out of scope:

1. Distributed database design.
2. Real payment gateway integration.
3. UI design.
4. Authentication and authorization.
5. Real-time hardware sensor integration.

Clarifying Questions Checklist

Use these questions before designing:

- How many users or objects should the system support?
- Is this a single-machine design or a distributed system?
- Should data be persisted, or is in-memory storage enough?
- Are there multiple user roles?
- Are there different object types?
- Can one user perform multiple roles?
- Is concurrency expected?
- Are cancellations, refunds, retries, or failures in scope?
- Should the system maintain history or audit logs?
- Are notifications required?
- Is pricing or ranking strategy fixed or configurable?
- Should we design APIs or only classes?
- Should we include database schema?

3. Constraints and Assumptions

Constraints limit the design. Assumptions fill missing information.

Constraints

Examples:

1. The system should support multiple floors.
2. A vehicle can occupy only one parking spot.
3. A parking spot can hold only one vehicle at a time.
4. A ticket is generated only when a spot is assigned.
5. Payment must be completed before exit.

Assumptions

Examples:

1. We are designing an in-memory system for interview simplicity.
2. The system runs in a single process unless concurrency is explicitly discussed.
3. Payment gateway integration is mocked through a `PaymentService` interface.
4. Admin setup APIs are not the primary focus.

Interview Tip

Say assumptions clearly. This shows maturity. A wrong assumption can be corrected early, but an unstated assumption can make the whole design look confused.

4. Actors and Use Cases

Actors are people, systems, or components that interact with your system.

Examples:

- User
- Admin
- Customer
- Driver
- Rider
- Librarian
- Parking Attendant
- Payment Gateway
- Notification Service
- Inventory System

Template:

Actors:

1. ...
2. ...

Use cases:

1. ...
2. ...

Example for Parking Lot:

Actors:

1. Vehicle owner
2. Parking attendant
3. Admin
4. Payment system

Use cases:

1. Park vehicle
2. Generate ticket
3. Find available spot
4. Calculate fee
5. Accept payment
6. Exit vehicle
7. Add or remove parking spots

5. Identify Core Entities

Entities are the main nouns in the problem. They usually become classes.

How to Find Entities

Read the problem statement and underline nouns.

Example:

Design a parking lot where vehicles can park in parking spots across multiple floors. A ticket is generated at entry and payment is collected at exit.

Possible entities:

- ParkingLot
- ParkingFloor
- ParkingSpot
- Vehicle
- Ticket
- Payment

- Gate
- FeeCalculator

Entity Checklist

For each entity, ask:

- Does it have state?
- Does it have behavior?
- Does it have identity?
- Does it manage other objects?
- Is it a value object?
- Is it an enum?
- Is it a service instead of an entity?

Common Entity Types

Domain Entity

An object with identity and lifecycle.

Examples:

- User
- Order
- Vehicle
- Ticket
- Booking
- Account

Value Object

An object defined by its values, usually immutable.

Examples:

- Money
- Address
- DateRange
- Location
- TimeSlot

Enum

A fixed set of values.

Examples:

```
enum VehicleType {  
    BIKE,
```

```
CAR,  
TRUCK  
}  
  
enum TicketStatus {  
    ACTIVE,  
    PAID,  
    LOST,  
    CANCELLED  
}
```

Service

A class that performs operations but usually does not represent a real-world object.

Examples:

- PaymentService
- NotificationService
- FeeCalculator
- MatchingService
- SearchService

Repository

A class responsible for storing and retrieving objects.

Examples:

- UserRepository
- TicketRepository
- BookingRepository

6. Identify Relationships

Relationships describe how classes are connected.

Common Relationship Types

Association

One class uses or knows about another class.

Example:

A Ticket is associated with a Vehicle.

Aggregation

One class contains another, but the child can exist independently.

Example:

A Department has Employees, but an Employee can exist without that Department.

Composition

One class owns another strongly. If the parent is destroyed, the child usually does not exist independently.

Example:

A ParkingLot contains ParkingFloors.

Inheritance

One class extends another.

Example:

```
abstract class Vehicle {  
    String licenseNumber;  
    VehicleType type;  
}  
  
class Car extends Vehicle {  
}
```

Dependency

One class temporarily depends on another to perform an operation.

Example:

ParkingService depends on FeeCalculator to calculate the parking fee.

Relationship Template

Relationships:

1. ParkingLot has multiple ParkingFloors.
2. ParkingFloor has multiple ParkingSpots.
3. ParkingSpot can hold one Vehicle.
4. Ticket belongs to one Vehicle and one ParkingSpot.

5. Payment belongs to one Ticket.
6. ParkingService uses SpotAssignmentStrategy and FeeCalculationStrategy.

7. Assign Responsibilities

This is one of the most important parts of LLD. A class should have a clear responsibility.

Responsibility Checklist

For each class, ask:

- What does this class know?
- What does this class do?
- What should this class not do?
- Does this class have too many reasons to change?
- Is this behavior better placed in another service?
- Can this class be tested independently?

Example Responsibility Table

Class	Responsibility
ParkingLot	Owns floors and coordinates high-level parking lot data
ParkingFloor	Manages spots on a floor
ParkingSpot	Represents a single physical parking spot
Vehicle	Represents a vehicle entering the lot
Ticket	Tracks parking session details
ParkingService	Handles park and exit workflows
FeeCalculator	Calculates fee based on ticket and pricing policy
PaymentService	Handles payment processing
SpotAssignmentStrategy	Chooses the best available spot

Interview Tip

Avoid putting all logic inside one manager class. Interviewers often reject designs where one class becomes a "God class".

8. Define Classes

For each class, define:

- Class name
- Responsibility
- Fields
- Constructor

- Public methods
- Private helper methods if needed
- Relationships with other classes

Class Template

```
class ClassName {  
    // Fields  
    private Type fieldName;  
  
    // Constructor  
    public ClassName(Type fieldName) {  
        this.fieldName = fieldName;  
    }  
  
    // Public methods  
    public ReturnType methodName(InputType input) {  
        // logic  
    }  
}
```

Detailed Class Description Template

Class: ...

Responsibility:
...

Fields:
1. ...
2. ...

Methods:
1. ...
2. ...

Relationships:
1. ...
2. ...

Example

```
class ParkingSpot {  
    private final String spotId;  
    private final ParkingSpotType spotType;  
    private boolean available;  
    private Vehicle parkedVehicle;
```

```
public ParkingSpot(String spotId, ParkingSpotType spotType) {
    this.spotId = spotId;
    this.spotType = spotType;
    this.available = true;
}

public boolean canFitVehicle(Vehicle vehicle) {
    return available && spotType.canFit(vehicle.getType());
}

public void parkVehicle(Vehicle vehicle) {
    if (!canFitVehicle(vehicle)) {
        throw new IllegalStateException("Vehicle cannot be parked in this
spot");
    }
    this.parkedVehicle = vehicle;
    this.available = false;
}

public void removeVehicle() {
    this.parkedVehicle = null;
    this.available = true;
}
}
```

9. Define APIs

APIs describe how external clients interact with your system. In interviews, APIs can mean REST endpoints or service methods.

Service API Template

```
class ParkingService {
    public Ticket parkVehicle(Vehicle vehicle);

    public PaymentReceipt exitVehicle(String ticketId, PaymentMode paymentMode);

    public List<ParkingSpot> getAvailableSpots(VehicleType vehicleType);
}
```

REST API Template

```
POST /vehicles/entry
Request:
{
  "licenseNumber": "KA-01-AB-1234",
  "vehicleType": "CAR",
  "entryGateId": "G1"
```

```
}

Response:
{
  "ticketId": "T123",
  "spotId": "F1-C12",
  "entryTime": "2026-07-08T10:30:00"
}
```

```
POST /vehicles/exit
Request:
{
  "ticketId": "T123",
  "paymentMode": "UPI"
}

Response:
{
  "receiptId": "R456",
  "amount": 80,
  "status": "SUCCESS"
}
```

API Design Checklist

- What is the input?
- What is the output?
- What errors can occur?
- Is the operation idempotent?
- Does the API change state?
- Does it need authentication?
- What validation is required?
- What happens if the object does not exist?

10. Data Structures

In LLD interviews, explain why you picked a data structure.

Common examples:

Need	Possible Data Structure
Fast lookup by id	HashMap
Maintain insertion order	LinkedHashMap
Sorted access	TreeMap or PriorityQueue
Queue of requests	Queue

Need	Possible Data Structure
Undo or backtracking	Stack
Avoid duplicates	HashSet
Graph relationships	Adjacency List
Nearest available item	PriorityQueue
Board games	2D array or matrix

Example:

```
I will use a HashMap<String, Ticket> for ticket lookup because exit flow needs fast lookup by ticket id.
```

11. Design Patterns

Use design patterns only when they genuinely simplify the design.

Strategy Pattern

Use when an algorithm can vary.

Examples:

- FeeCalculationStrategy
- SpotAssignmentStrategy
- DiscountStrategy
- MatchingStrategy
- PricingStrategy

```
interface FeeCalculationStrategy {
    Money calculateFee(Ticket ticket);
}

class HourlyFeeCalculationStrategy implements FeeCalculationStrategy {
    public Money calculateFee(Ticket ticket) {
        // calculate hourly fee
        return new Money(100);
    }
}
```

Factory Pattern

Use when object creation varies by type.

Examples:

- VehicleFactory
- PaymentFactory
- NotificationFactory
- PieceFactory in chess

```
class VehicleFactory {  
    public Vehicle createVehicle(String licenseNumber, VehicleType type) {  
        switch (type) {  
            case CAR:  
                return new Car(licenseNumber);  
            case BIKE:  
                return new Bike(licenseNumber);  
            default:  
                throw new IllegalArgumentException("Unsupported vehicle type");  
        }  
    }  
}
```

Observer Pattern

Use when multiple subscribers should be notified of an event.

Examples:

- Notification system
- Order status updates
- Stock price updates
- Ride status updates

Singleton Pattern

Use carefully. It is common in interviews but can make testing harder.

Possible use:

- Configuration manager
- Logger

Tradeoff:

Singleton ensures one shared instance, but it introduces global state and can make unit testing harder.

State Pattern

Use when behavior changes based on state.

Examples:

- Vending Machine
- ATM
- Order lifecycle
- Ticket lifecycle

Command Pattern

Use when requests should be represented as objects.

Examples:

- Undo/redo system
- Remote control
- Task queue

12. Apply SOLID Principles

SOLID is important in placement LLD interviews. Mention it naturally while explaining the design.

S - Single Responsibility Principle

A class should have only one reason to change.

Bad:

```
Ticket class calculates fee, processes payment, sends notification, and stores ticket data.
```

Better:

```
Ticket stores ticket data.  
FeeCalculator calculates fee.  
PaymentService processes payment.  
NotificationService sends notification.
```

Interview line:

```
I separated fee calculation and payment processing so that Ticket remains focused on representing a parking session.
```

O - Open/Closed Principle

Classes should be open for extension but closed for modification.

Example:

Instead of changing `ParkingService` every time a new pricing rule is added, I use `FeeCalculationStrategy`.

L - Liskov Substitution Principle

Subclasses should be usable wherever the parent class is expected.

Example:

If `Car` and `Bike` extend `Vehicle`, methods accepting `Vehicle` should work correctly for both.

Avoid:

A subclass that throws unsupported operation exceptions for normal parent behavior.

I - Interface Segregation Principle

Do not force classes to implement methods they do not need.

Bad:

```
interface Machine {  
    void print();  
    void scan();  
    void fax();  
}
```

Better:

```
interface Printer {  
    void print();  
}  
  
interface Scanner {  
    void scan();  
}
```

D - Dependency Inversion Principle

High-level classes should depend on abstractions, not concrete classes.

Example:

```
class ParkingService {
    private final FeeCalculationStrategy feeCalculationStrategy;
    private final PaymentService paymentService;

    public ParkingService(
        FeeCalculationStrategy feeCalculationStrategy,
        PaymentService paymentService
    ) {
        this.feeCalculationStrategy = feeCalculationStrategy;
        this.paymentService = paymentService;
    }
}
```

Interview line:

ParkingService depends on FeeCalculationStrategy instead of a concrete HourlyFeeCalculator, so we can plug in different pricing rules.

13. Edge Cases

Edge cases show that you can think beyond the happy path.

General Edge Case Checklist

- Invalid input
- Null or missing fields
- Duplicate requests
- Object not found
- Object already exists
- Resource unavailable
- Payment failure
- Concurrent booking or allocation
- Cancellation
- Timeout
- Partial failure
- Retry handling
- Permission denied
- State transition not allowed
- Capacity full
- Empty system
- Boundary values
- Race conditions
- External service unavailable

Parking Lot Edge Cases

- Parking lot is full.
- No spot available for a specific vehicle type.
- Ticket id is invalid.
- Ticket is already paid.
- Vehicle tries to exit without payment.
- Payment fails.
- Same vehicle tries to enter twice.
- Gate is inactive.
- Spot assignment fails after ticket creation.
- Two vehicles try to book the same spot at the same time.

BookMyShow Edge Cases

- Seat is already booked.
- Seat is temporarily locked by another user.
- Payment fails after seat lock.
- User refreshes payment page.
- Seat lock expires.
- Show is cancelled.
- Booking is cancelled.
- Multiple users try to book the same seat.

Splitwise Edge Cases

- Expense amount is zero or negative.
- User is not part of the group.
- Split percentages do not add up to 100.
- Exact split amounts do not add up to total.
- User deletes account with pending balances.
- Currency mismatch.

Elevator Edge Cases

- Elevator is overloaded.
- Door is stuck.
- Emergency stop is pressed.
- Multiple requests come at the same time.
- Elevator is under maintenance.
- No elevator is available.
- Direction changes should be handled correctly.

14. Extension Points

Extension points are places where the system can grow without major code changes.

Mentioning extension points makes your design look future-ready.

Examples:

1. New vehicle types can be added by extending `Vehicle` and updating `VehicleType`.
2. New pricing strategies can be added by implementing `FeeCalculationStrategy`.
3. New payment modes can be added by implementing `PaymentProcessor`.
4. New notification channels can be added by implementing `NotificationSender`.
5. New spot assignment algorithms can be added by implementing `SpotAssignmentStrategy`.

Extension Point Checklist

- New user roles
- New object types
- New payment modes
- New pricing rules
- New notification channels
- New search filters
- New sorting rules
- New matching algorithms
- New state transitions
- New storage backends
- New external integrations

15. Tradeoffs

Tradeoffs show that you understand design decisions are not free.

Common Tradeoffs

Inheritance vs Composition

Inheritance is useful when there is a strong "is-a" relationship, but it can create rigid hierarchies. Composition is more flexible because behavior can be plugged in through objects.

Interface vs Concrete Class

Interfaces improve extensibility and testability, but too many interfaces can make a small design unnecessarily complex.

In-Memory Storage vs Database

In-memory storage is simple for interviews and unit tests, but it loses data after restart. A database is needed for persistence and real-world usage.

Synchronous vs Asynchronous Processing

Synchronous processing is simple and gives immediate results. Asynchronous processing improves scalability for slow tasks like notifications, but it adds complexity.

Strong Consistency vs Availability

Strong consistency avoids double booking and incorrect state, but it may reduce availability under failures. For booking and payment systems, consistency is usually more important.

Simple Design vs Highly Extensible Design

A simple design is easier to understand and implement. A highly extensible design supports future changes but can introduce extra abstractions.

Interview Tip

Do not just say "this is scalable". Explain what decision makes it scalable or extensible.

16. Concurrency

Concurrency matters when multiple users can modify the same resource.

Examples:

- Two users booking the same seat.
- Two vehicles getting the same parking spot.
- Two users withdrawing from the same bank account.
- Multiple elevator requests arriving at the same time.

Concurrency Handling Options

1. Synchronize critical sections.
2. Use locks at resource level.
3. Use optimistic locking with version numbers.
4. Use database transactions.

5. Use seat or resource locking with expiry.
6. Use queues for ordered processing.

Interview Line

For seat booking, I would lock the seat for a short duration during payment. If payment is not completed before expiry, the seat becomes available again.

17. State Management

Many LLD problems have objects that move through states.

Examples:

```
Ticket: ACTIVE -> PAID -> CLOSED
Order: CREATED -> CONFIRMED -> SHIPPED -> DELIVERED -> CANCELLED
Booking: INITIATED -> LOCKED -> CONFIRMED -> CANCELLED -> EXPIRED
ATM: IDLE -> CARD_INSERTED -> PIN_VERIFIED -> TRANSACTION_SELECTED ->
CASH_DISPENSED
```

State Transition Checklist

- What are the possible states?
- What events cause state changes?
- Which transitions are invalid?
- Who is allowed to trigger the transition?
- Should state history be stored?
- What happens on failure?

18. Validation

Validation keeps bad data out of the system.

Common validations:

- Required fields should not be null.
- Amount should be positive.
- End time should be after start time.
- User should exist.
- Resource should exist.
- User should have permission.
- State should allow the operation.
- Capacity should not be exceeded.

Example:

```
public void addExpense(Expense expense) {
    if (expense.getAmount().isNegativeOrZero()) {
        throw new IllegalArgumentException("Expense amount must be positive");
    }
    if (expense.getParticipants().isEmpty()) {
        throw new IllegalArgumentException("Expense must have participants");
    }
}
```

19. Error Handling

Define meaningful errors.

Examples:

```
class SpotNotAvailableException extends RuntimeException {
}

class InvalidTicketException extends RuntimeException {
}

class PaymentFailedException extends RuntimeException {
}
```

Error Handling Checklist

- Use clear exception names.
- Do not silently ignore failures.
- Do not expose internal errors directly to users.
- Keep business errors separate from system errors.
- Think about retries for external service failures.

20. Testing Strategy

Mentioning tests is a strong placement signal.

Unit Tests

Test individual classes.

Examples:

- FeeCalculator calculates correct fee.
- ParkingSpot accepts correct vehicle type.
- Split strategy divides amounts correctly.
- Elevator chooses correct next floor.

Integration Tests

Test workflows across classes.

Examples:

- Park vehicle -> generate ticket -> pay -> exit.
- Select seat -> lock seat -> pay -> confirm booking.
- Add expense -> split amount -> update balances.

Edge Case Tests

Examples:

- Parking lot full.
- Payment failure.
- Duplicate booking.
- Invalid ticket id.
- Cancelled order cannot be shipped.

21. Complete Answer Skeleton

Use this skeleton during interviews.

```
1. I will first clarify the requirements.

2. Functional requirements:
  - ...
  - ...

3. Non-functional requirements:
  - ...
  - ...

4. Assumptions:
  - ...
  - ...

5. Core entities:
  - ...
  - ...

6. Relationships:
  - ...
  - ...

7. Main classes and responsibilities:
  - ...
  - ...

8. APIs:
  - ...
  - ...
```

9. Design patterns:

- ...
- ...

10. SOLID principles:

- ...
- ...

11. Edge cases:

- ...
- ...

12. Extension points:

- ...
- ...

13. Tradeoffs:

- ...
- ...

22. Sample Mini Design: Parking Lot

Requirements

1. Support multiple floors.
2. Support different vehicle types.
3. Assign a suitable parking spot.
4. Generate a ticket at entry.
5. Calculate fee at exit.
6. Accept payment.
7. Free the spot after successful exit.

Entities

1. ParkingLot
2. ParkingFloor
3. ParkingSpot
4. Vehicle
5. Ticket
6. Payment
7. EntryGate
8. ExitGate
9. FeeCalculator
10. SpotAssignmentStrategy

Classes

```
enum VehicleType {
    BIKE,
    CAR,
    TRUCK
}

enum ParkingSpotType {
    BIKE_SPOT,
    COMPACT,
    LARGE
}

abstract class Vehicle {
    private final String licenseNumber;
    private final VehicleType vehicleType;

    protected Vehicle(String licenseNumber, VehicleType vehicleType) {
        this.licenseNumber = licenseNumber;
        this.vehicleType = vehicleType;
    }

    public String getLicenseNumber() {
        return licenseNumber;
    }

    public VehicleType getVehicleType() {
        return vehicleType;
    }
}

class Car extends Vehicle {
    public Car(String licenseNumber) {
        super(licenseNumber, VehicleType.CAR);
    }
}

class Ticket {
    private final String ticketId;
    private final Vehicle vehicle;
    private final ParkingSpot parkingSpot;
    private final long entryTime;
    private Long exitTime;
    private TicketStatus status;

    public Ticket(String ticketId, Vehicle vehicle, ParkingSpot parkingSpot, long
entryTime) {
        this.ticketId = ticketId;
        this.vehicle = vehicle;
        this.parkingSpot = parkingSpot;
        this.entryTime = entryTime;
        this.status = TicketStatus.ACTIVE;
    }
}
```

```
public void markPaid(long exitTime) {  
    this.exitTime = exitTime;  
    this.status = TicketStatus.PAID;  
}  
}
```

Service Layer

```
class ParkingService {  
    private final SpotAssignmentStrategy spotAssignmentStrategy;  
    private final FeeCalculationStrategy feeCalculationStrategy;  
    private final PaymentService paymentService;  
    private final TicketRepository ticketRepository;  
  
    public ParkingService(  
        SpotAssignmentStrategy spotAssignmentStrategy,  
        FeeCalculationStrategy feeCalculationStrategy,  
        PaymentService paymentService,  
        TicketRepository ticketRepository  
    ) {  
        this.spotAssignmentStrategy = spotAssignmentStrategy;  
        this.feeCalculationStrategy = feeCalculationStrategy;  
        this.paymentService = paymentService;  
        this.ticketRepository = ticketRepository;  
    }  
  
    public Ticket parkVehicle(Vehicle vehicle) {  
        ParkingSpot spot = spotAssignmentStrategy.findSpot(vehicle);  
        spot.parkVehicle(vehicle);  
  
        Ticket ticket = new Ticket(generateTicketId(), vehicle, spot,  
            System.currentTimeMillis());  
        ticketRepository.save(ticket);  
        return ticket;  
    }  
  
    public PaymentReceipt exitVehicle(String ticketId, PaymentMode paymentMode) {  
        Ticket ticket = ticketRepository.findById(ticketId)  
            .orElseThrow(InvalidTicketException::new);  
  
        Money fee = feeCalculationStrategy.calculateFee(ticket);  
        PaymentReceipt receipt = paymentService.pay(fee, paymentMode);  
  
        ticket.markPaid(System.currentTimeMillis());  
        ticket.getParkingSpot().removeVehicle();  
  
        return receipt;  
    }  
  
    private String generateTicketId() {
```

```
        return "TICKET-" + System.nanoTime();  
    }  
}
```

SOLID Explanation

1. Single Responsibility:

Ticket stores ticket data, FeeCalculationStrategy calculates fees, and PaymentService handles payment.

2. Open/Closed:

New fee rules can be added by creating a new FeeCalculationStrategy implementation.

3. Dependency Inversion:

ParkingService depends on strategy and service interfaces rather than concrete implementations.

4. Interface Segregation:

Payment, notification, and fee calculation are separate interfaces.

5. Liskov Substitution:

Car, Bike, and Truck can be used anywhere a Vehicle is expected.

Tradeoffs

1. I used Strategy Pattern for fee calculation because pricing rules can change.
2. I used in-memory repositories for simplicity, but in production I would use a database.
3. I kept Vehicle inheritance simple. If vehicle behavior grows, composition may be better.
4. I separated ParkingService from ParkingSpot so the workflow logic does not overload the entity class.

Edge Cases

1. Parking lot is full.
2. No compatible spot is available.
3. Invalid ticket id.
4. Ticket already paid.
5. Payment failure.
6. Concurrent spot allocation.
7. Vehicle tries to enter twice.

23. Quick LLD Interview Checklist

Before finishing your answer, check:

- Did I clarify the scope?
- Did I list functional requirements?
- Did I mention non-functional requirements?
- Did I define entities?
- Did I explain relationships?
- Did I assign class responsibilities?
- Did I avoid one class doing everything?
- Did I define APIs or service methods?
- Did I mention useful design patterns?
- Did I apply SOLID principles?
- Did I discuss edge cases?
- Did I mention extension points?
- Did I explain tradeoffs?
- Did I think about concurrency?
- Did I mention testing?

24. Useful Interview Phrases

Use these lines while explaining:

I will keep this responsibility separate so the class has only one reason to change.

I will depend on an interface here because the implementation can vary.

This is an extension point. If a new pricing rule is added, we can plug in a new strategy.

For interview simplicity, I am using in-memory storage, but this can be replaced with a database-backed repository.

The main concurrency risk is double allocation, so the resource assignment should be atomic.

I am choosing composition here because the behavior may change independently at runtime.

This design favors readability and extensibility over premature optimization.

25. Final Rule

A good LLD answer is not just a list of classes. It should explain:

- Why these classes exist.
- What each class is responsible for.
- How objects interact.
- What can go wrong.
- How the design can grow.
- What tradeoffs you made.

If you can explain these clearly, your LLD answer will look structured, practical, and placement-ready.